Agentic Coding for Economists

Day 2: Research Pipeline and Spec-Driven Development

Participant repo: `https://github.com/meleantonio/AC4E_Pavia`

Antonio Mele
University of Pavia

June 22–24, 2026

Day 1 Recap

- **Agent workflows**: read-only, planning, implementation, debugging
- **Lanes**: chosen tool lane and main research-language lane
- **Governance**: `AGENTS.md`, privacy boundaries, first branch or PR path
- **Project framing**: `docs/project_brief.md` and an initial backlog seed

Day 2 starts from: a scoped project idea, not a full implementation plan.

Day 2 Goals

1. **Spec-driven development**: turn the brief into intent, requirements, design, and tasks
2. **Research design memo**: separate confirmed facts, assumptions, blockers, and limitations
3. **Literature and inputs**: build a literature map plus a data or model-input map
4. **Acceptance criteria**: convert vague work into observable checks and prioritized issues
5. **Context patterns**: practice tight, wide, and external context; use `/sdd` during Sprint A

Agent Skills

- **What: Agent Skills** are reusable instructions (typically `SKILL.md`) that tell the chosen agent *when* and *how* to run a workflow.
- **Attach / invoke:** Type `/sdd` (or the skill name), attach the skill in chat, or let the agent match the skill `description` in frontmatter.
- **Scope: Project** skills live in-repo (e.g. `.cursor/skills/`, `.claude/skills/`, `.agents/skills/`); **user** skills live in your home directory for cross-project reuse.
- **This course:** Bundled skills under `skills/course/` (SDD, literature-mapper, replication-checker). Day 3 goes deeper on *authoring* and combining skills with subagents.

SDD via /sdd

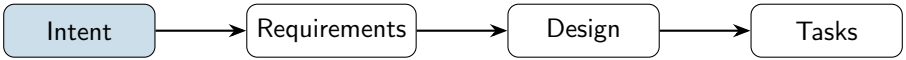
Live demo: attach the instructor SDD skill, then run intent → requirements → design → tasks.

Project layout (after `init` or manual scaffold):

```
spec/
  intent.md          # research question, audience, success criteria
  requirements.md    # EARS-style REQ-* statements
  design.md          # data flow, estimators, folder map
  tasks.md           # ordered issues with acceptance criteria
steering/
  coding-standards.md
```

Prompts: "Use /sdd init" → "/sdd reqs" → "/sdd design" → "/sdd tasks". Alternatively invoke `sdd-orchestrator`.

SDD Lifecycle



What `/sdd` **automates:** scaffolding `spec/`, drafting EARS requirements from intent, translating requirements into design and tasks, and `status` checks.

What you still own as economist:

- **Intent** — research question, contribution, and what claim the project should support
- **Requirements** — identification assumptions, data contracts, replication rules you will defend
- **Design** — estimator or model choices, robustness plan, software map
- **Tasks** — prioritized issues with acceptance criteria you can verify

EARS: Easy Approach to Requirements Syntax

What it is: A lightweight pattern language (common in safety-critical and systems engineering) to write **unambiguous, testable** requirements. In research code, it reduces "I thought you meant..." failures.

Modal verbs: `SHALL` = mandatory; `SHOULD` = recommended; `MAY` = optional.

Common patterns (combine with SHALL for checks you can automate or review):

- **WHEN** *trigger*, the system SHALL *response* — e.g. WHEN a clean run starts, the pipeline SHALL exit 0 and write `outputs/manifest.json`.
- **WHERE** *constraint*, the system SHALL *behavior* — e.g. WHERE data are missing at random by design, the estimator SHALL document the selection rule.
- **IF** *precondition*, THEN the system SHALL *response* — e.g. IF the user passes `-robustness`, THEN the script SHALL also run the placebo specification.
- **GIVEN** *context*, the system SHALL *observable* — e.g. GIVEN fixed random seed 42, simulation outputs SHALL match reference moments within tolerance ε.

From Intent to EARS: three flavors of economics

1. Applied empirical (panel / causal) — Intent: causal effect of policy X on Y. Requirements: reproducible panel, documented treatment timing, pre-specified estimators. Design: clean $\rightarrow$ DiD/RD/IV $\rightarrow$ tables.

2. Applied theory / structural (micro) — Intent: characterize equilibrium or comparative statics in a market with search/frictions. Requirements: fixed parameter grid; Bellman iterations converge; equilibrium conditions residual $< \varepsilon$. Design: state space $\rightarrow$ value/policy iteration $\rightarrow$ simulated moments vs data.

3. Macro (DSGE / HANK pipeline) — Intent: quantify transmission of monetary/fiscal shock. Requirements: model block order; calibration targets (moments) matched; IRFs stored with metadata (shock size, horizon). Design: mod file or Python $\rightarrow$ solve SS $\rightarrow$ perturbation $\rightarrow$ IRFs/variance decomposition.

Mapping SDD to Economics Research

SDD stage	Economics artifact
Intent	Research question, contribution, scope (empirical vs theory vs quantitative macro)
Requirements	Reproducibility rules, data contracts, numerical tolerances, output schemas
Design	Data flow or model blocks; estimation vs simulation; robustness plan
Tasks	Issue backlog (data, model code, estimation, figures, replication)

Day 2 deliverables (full SDD chain; artifacts may live in **separate files** linked from the hub memo):

- **Intent** — e.g. `docs/intent.md` or the Intent block in `docs/research_design_memo.md`
- **Requirements** — numbered REQ-* in memo or `docs/requirements.md` (EARS-style)
- **Design** — pipeline/model diagram or `docs/design.md` (data + code map)
- **Tasks** — GitHub issues (or `docs/tasks.md`) with acceptance criteria
- **Hub**: `docs/research_design_memo.md` summarizes and *links* to the above when they grow large

SDD Example: Hub Memo (+ optional linked files)

A single memo can hold everything *or* point to heavier artifacts.

```
# Research Design: Minimum Wage and Fast-Food Employment

## Linked artifacts (optional)
# docs/requirements.md - full EARS REQ list
# docs/design.md - data flow + estimator choices
# docs/tasks.md - issue IDs and milestones

## Intent
Produce a transparent baseline comparison for NJ vs eastern PA
fast-food employment after the April 1992 minimum wage increase.

## Requirements (excerpt)
- [REQ-1] WHEN replication runs, SHALL reproduce tables/figures (hash or diff)
- [REQ-2] GIVEN public Card-Krueger inputs, SHALL report sample counts
          by state and wave before estimating any baseline comparison

## Design
1. Public data -> documented restaurant-level analysis file
2. NJ treatment group; eastern PA comparison group
3. Baseline difference-in-differences table + limitations

## Tasks (prioritized)
- #1 Data/model-input map  #2 Cleaning script  #3 Baseline table
- #4 Replication README
```

Research Design Memo

Bridge: the memo turns a project brief into implementation-ready research constraints.

```
# Research Design Memo

## Research question
## Motivation and literature
## Hypotheses, model claims, or object of interest
## Data or model inputs
## Method
## Identification, assumptions, or equilibrium conditions
## Outputs
## Limitations
## Replication implications
```

Review prompt: "Act as a skeptical economist. Flag vague claims, missing definitions, weak identification statements, unclear assumptions, and outputs that cannot be verified."

Tight Context Patterns

Pattern 1: Single-file scope (empirical)

> "Only edit `src/estimation/did.py`: add cluster-robust SEs at the county level."

Pattern 2: Single-file scope (theory / structural)

> "Only edit `src/model/bellman.py`: vectorize the value-function iteration; do not change `parameters.yaml`."

Pattern 3: Folder scope

> "With `references/` in context: check citation consistency with our BibTeX."

Pattern 4: Explicit exclusion (macro pipeline)

> "Modify only `scripts/run_dsge_irf.py`; do not touch the `models/` mod files until calibration is frozen."

Artifact: record useful prompts and tool-lane caveats in `notes/context_patterns.md`.

Tight, Wide, and External Context

Context	Use when	Example
Tight	One file, one function, one log, one memo	"Only read this Stata log; identify the first error."
Wide	Names or claims must be consistent across files	"Search for every outcome-variable definition."
External	Official docs, data metadata, or literature matter	"Use official API docs; include links and caveats."

For current product UI, official docs beat old slides. Add a "verify in your version" note where labels or fields vary.

Web search for literature

Use cases (ask explicitly; exact @ menu varies):

- "Search the web: recent DiD papers on minimum wage (2020–2025)?"
- "Search the web: existence proofs or algorithms for equilibrium in random search models with heterogeneous firms?"
- "Search the web: Dynare vs Julia for medium-scale DSGE with financial frictions — practitioner comparisons?"
- "Search the web: BibTeX for [Author (Year)]; journal abbreviation consistent with AER style."

Prompt scaffolding: "BibTeX + 2–3 sentence summary; for theory, state assumptions; for macro, note calibration vs estimation."

Docs for API References

Use cases:

- **"Use the docs for `statsmodels` / `linearmodels`: two-way FE with clustered SEs"**
- **"Use the docs for `scipy.optimize`: root-finding for steady-state nonlinear equations"**
- **"Use the docs for `jax` or `numba`: JIT for repeated value-function updates"**
- **"Use the docs for `pandas merge_asof` for real-time macro data alignment"**

Reduces hallucination: Ground implementation in actual API signatures and examples.

Literature Mapping Workflow

1. **Seed:** 3–5 key papers you already know (empirical anchor, canonical theory, macro benchmark)
2. **Web search:** "Papers on [topic] similar to [Author (Year)]"; for theory, add "assumptions + equilibrium concept"
3. **BibTeX:** Ask for entries; verify keys and fields
4. **Structure:** Themes — e.g. identification / equilibrium existence / solution algorithms / calibration targets
5. **Summary table:** Paper | Method / model class | Data or moments | Main result

Artifact: `references/bibliography.bib` **with 15–25 entries**

BibTeX Generation

```
@article{card1994minimum,
  author = {Card, David and Krueger, Alan B.},
  title = {Minimum Wages and Employment},
  journal = {American Economic Review},
  volume = {84},
  number = {4},
  pages = {772--793},
  year = {1994}
}
```

Prompt: "Generate BibTeX for [paper]; use `author_year:keyword` key format. Include DOI if available." **Verify**: Check journal abbreviations, page ranges, author names.

Literature Review Structure

- **Section 1:** Research question and contribution (empirical, structural, or quantitative macro)
- **Section 2:** Prior work (thematic grouping) — examples:
  - **Causal designs (IV, DiD, RDD) *or* equilibrium/existence results *or* DSGE variants**
  - **Data / measurement *or* calibration targets *or* stylized facts**
  - **Robustness / sensitivity *or* alternative solution methods**
- **Gap:** What we add (new data, new mechanism, new shock, new quantitative channel)

AI assist: "Attach `references/bibliography.bib`; draft a 1-page literature section; tag each cite as Empirical / Theory / Macro."

Data or Model-Input Map

Empirical projects

- **Source**: where data come from
- **Access**: public, restricted, API, manual, proprietary
- **Unit**: restaurant, firm, person, country-year, region-month
- **Variables**: outcome, treatment, controls, IDs, time
- **Coverage and restrictions**: years, geography, license, confidentiality

Theory, macro, or simulation projects

- **Primitives**: preferences, technology, information
- **Parameters**: value, source, calibration target
- **Equations**: model block or equilibrium condition
- **Solver inputs**: grid, tolerance, initial condition, seed
- **Checks**: feasibility, market clearing, convergence

Artifact: `docs/data_source_map.md` or equivalent model-input map.

Vague Task → Acceptance Criteria

Why this slide matters: Agents (and humans) optimize for the prompt they see. If "done" is undefined, you get plausible-looking code that fails your real standard. **Acceptance criteria** turn intent into checks: inputs, outputs, formats, runtime, and edge cases.

Recipe: Start from the **smallest observable artifact** (a file, a test, a numeric tolerance). Add **negative cases** (empty panel, wrong schema). Tie each criterion to a **verification command** (`pytest`, `diff`, `python script.py -dry-run`).

Before (too vague):

> "Build the estimation script."

After (acceptance criteria):

- Reads `data/panel.csv`; fails fast with clear error if columns missing
- Writes coefficients + SEs to `outputs/did_results.csv` with documented column order
- Cluster-robust SEs at county level; seed fixed for bootstrap if used
- Run completes in < 2 minutes on reference machine; logged in README

Acceptance Criteria: Example A (empirical pipeline)

Vague: "Clean the FRED series and merge with Compustat."

Criteria:

- **Inputs:** Raw CSV under `data/raw/` with schema in `docs/data_dictionary.md`
- **Outputs:** `data/clean/firm_quarter.csv`; row count and date range printed to log
- **Rules:** No duplicate (firm, quarter); missing codes propagated explicitly; merge keys documented
- **Check:** Running `scripts/validate_panel.py` exits 0 and matches gold row counts ± 0

Verify: Run validator; spot-check 5 random firms against source.

Acceptance Criteria: Example B (applied theory code)

Vague: "Solve the Bellman equation for the worker side."

Criteria:

- **State space:** Documented grid bounds and number of nodes; units consistent with `parameters.yaml`
- **Convergence:** Max sup-norm distance between successive value functions $< 10^{-5}$ (or iteration cap with warning)
- **Outputs:** `outputs/value_function.npy` + plot `outputs/vf_slice.png` for one parameter draw
- **Check:** Unit test on known closed-form special case (if available) or monotonicity checks

Verify: Run test suite; inspect convergence log.

Acceptance Criteria: Example C (macro DSGE / IRFs)

Vague: "Run the model and show impulse responses."

Criteria:

- **Steady state:** Residuals of static equations $< 10^{-8}$; solver log archived under `outputs/ss_log.txt`
- **IRF:** 40 quarters; monetary shock size documented (e.g. 25 bps); variables list fixed
- **Artifacts:** `outputs/irf_mp_shock.csv` + figure `outputs/irf_mp.pdf` with same style as paper
- **Reproducibility:** Random seed + software version (Dynare/Julia) in `README_replication.md`

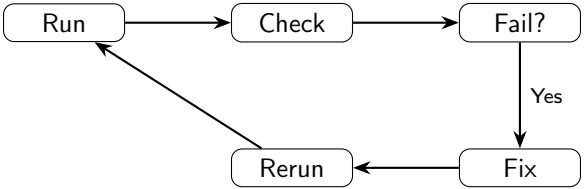
Verify: Re-run from clean `outputs/`; compare to checksum or prior run `diff` within tolerance.

Consistency Check Pattern

1. **Freeze** pipeline (no more edits to core scripts)
2. **Run** from clean state: `python scripts/run_pipeline.py`
3. **Compare** outputs to reference (or previous run)
4. **Fail** $\rightarrow$ fix $\rightarrow$ rerun until identical

Review agent: "Check that `outputs/` tables match `references/expected/`. Report discrepancies."

Fail-Fix-Rerun Cycle



Pass when: All outputs match; replication README documents exact command.

Definition of Done for Day 2

1. **Design memo:** `docs/research_design_memo.md` separates question, literature, inputs, method, assumptions, outputs, limitations, and replication implications.
2. **SDD chain:** intent, requirements, design, and tasks are documented in the memo, linked files, or GitHub issues.
3. **Literature map:** bibliography or reference list with relevance notes and uncertainty flagged.
4. **Data/model-input map:** empirical source map or model primitives/parameters/equations/solver checks.
5. **Prioritized issues:** allowed files, dependencies, acceptance criteria, verification evidence, and out-of-scope note.
6. **SDD skill use:** used `/sdd` or `sdd-orchestrator` for at least one lifecycle step; artifacts in `spec/` or linked from the memo.
7. **Harness plan:** one candidate custom skill or reusable workflow and one review role or subagent identified for Day 3.

Day 2 Deliverables

Checklist (full SDD chain):

- ☐ **Intent + requirements + design + tasks covered (`research_design_memo.md` ± linked docs)**
- ☐ `references/bibliography.bib` **or literature map with verified citation details**
- ☐ `docs/data_source_map.md` **or model-input map**
- ☐ **Backlog prioritized (labels + dependencies + acceptance criteria)**
- ☐ `notes/context_patterns.md` **with useful prompts and version caveats**
- ☐ **Used `/sdd` or `sdd-orchestrator` (or equivalent SDD skill) during Sprint A**
- ☐ **Candidate custom skill/workflow and reviewer role for Day 3**

Day 3 Preview

Orchestration, Autonomous Agents, Replication

- Skills and role prompts: reviewer, replicator, bibliographer, model checker
- Subagents, MCP, cloud agents, and swarms: use only with clear scope and review gates
- Autonomous-agent risk card before delegating across tools, memory, schedules, or cloud runners
- Final integration: replication README, presentation, and 30-day adoption plan

Deliverables: skill/subagent use, orchestration log, risk card, `replication/README.md`, 5–7 minute presentation, 30-day plan.